Compartición de secretos con códigos QR

El esquema de Shamir sobre $\mathbb{F}_{2^8}$, criptografía visual, algoritmos, librerías en Python y aplicaciones

Dr. José de Jesús Ángel Ángel
Facultad de Ingeniería, Universidad Anáhuac México

21 de octubre de 2026

Resumen

Un código QR es un medio físico barato, duradero y legible con cualquier teléfono; un esquema de umbral (k, n) reparte un secreto de modo que k partes lo reconstruyen y $k-1$ no revelan nada. Este documento estudia las dos formas de combinarlos. En la primera, cada parte de Shamir se *codifica* en un QR: se presenta el esquema sobre el campo $\mathbb{F}_{2^8}$, un formato textual de parte que aprovecha el modo alfanumérico del QR, algoritmos en pseudocódigo, una implementación completa en Python con generación y lectura de imágenes, y un modo híbrido para secretos largos. En la segunda, el propio QR *es* el secreto y se reparte como imagen mediante criptografía visual (XOR y el esquema de Naor y Shamir). Se cierra con librerías, herramientas existentes, aplicaciones reales y posibles, y recomendaciones de seguridad. Todo el código se ejecutó y las figuras se generaron con él.

Índice general

1 Motivación: dos maneras de juntar Shamir y QR **2**

2 Lo que hay que saber de un código QR **3**

3 Shamir sobre $\mathbb{F}_{2^8}$: un byte a la vez **3**
3.1 Por qué $\mathbb{F}_{2^8}$ . . . 3
3.2 Algoritmos . . . 4
3.3 Ejemplo a mano: un byte, umbral 2 de 3 . . . 5
3.4 Implementación en Python . . . 5

4 De la parte al código QR **7**
4.1 Qué debe llevar cada QR . . . 7
4.2 Implementación . . . 9
4.3 Ejemplo ilustrativo . . . 10
4.4 ¿Qué tan grande sale el QR? . . . 11

5 Secretos largos: el modo híbrido **11**

6 Cuando el QR es el secreto: criptografía visual **12**
6.1 La idea de Naor y Shamir . . . 12
6.2 Esquema (n, n) por XOR . . . 13
6.3 Esquema $(2, 2)$ de Naor y Shamir: superposición física . . . 13

6.4 Esquemas (k, n) y QR con portada 16

7 Librerías y herramientas **17**
7.1 Librerías de Python 17
7.2 Herramientas existentes 17

8 Aplicaciones **18**
8.1 Aplicaciones en uso 18
8.2 Aplicaciones posibles 18

9 Recomendaciones de seguridad **19**

Lista de algoritmos

1 Aritmética de $\mathbb{F}_{2^8}$ con tablas de exponentes y logaritmos 4
2 Reparto de Shamir byte a byte sobre $\mathbb{F}_{2^8}$ 5
3 Reconstrucción byte a byte por Lagrange en $x = 0$ 6
4 Emisión de las partes como códigos QR 8
5 Lectura de los QR y reconstrucción verificada 8
6 Criptografía visual (n, n) por XOR sobre la matriz de un QR 13
7 Criptografía visual $(2, 2)$ de Naor y Shamir 14

Índice de figuras

1 Las cinco partes de un reparto $(3, 5)$, nivel de corrección H. Cualesquiera tres reconstruyen el secreto. 11
2 La parte 3 intacta y con una mancha. Con nivel H sigue siendo legible. 11
3 Esquema $(3, 3)$ por XOR. Las sombras son ruido; su XOR reproduce el QR módulo a módulo y `zxing-cpp` lo lee. Con dos sombras el resultado sigue siendo ruido. . . . 13
4 Esquema $(2, 2)$ de Naor y Shamir aplicado a un QR. La superposición muestra el QR con contraste reducido; tras promediar bloques y umbralizar se recupera exactamente. 14

1 Motivación: dos maneras de juntar Shamir y QR

El esquema de Shamir [1] resuelve un problema de custodia: que ninguna persona tenga sola una clave valiosa, pero que un grupo suficiente pueda recuperarla. Queda un problema práctico: *¿en qué soporte se entrega cada parte?* Un archivo en una memoria USB se corrompe o se pierde, y una cadena hexadecimal de 70 caracteres copiada a mano invita a errores. Un código QR impreso, en cambio, se lee en segundos, tolera manchas gracias a su corrección de errores y no depende de ningún dispositivo particular.

Hay dos combinaciones con propósitos distintos:

A. El QR como *contenedor* de la parte.
Se reparte el secreto con Shamir y cada parte se escribe en un QR. El QR sólo es el medio de almacenamiento; la seguridad la aporta Shamir. Es la opción práctica y la que usan las herramientas existentes (Secciones 3 a 5).

B. El QR como *secreto*.
Se parte la *imagen* de un QR en "sombras" de ruido; sólo al combinarlas aparece un QR legible. Es criptografía visual: llamativa, útil en docencia y con una línea activa de investigación (Sección 6).

2 Lo que hay que saber de un código QR

El código QR está normalizado en ISO/IEC 18004 [3]. Para nuestro propósito bastan cuatro hechos.

Versiones y tamaño. Hay 40 versiones. La versión v es una matriz de $(17 + 4v) \times (17 + 4v)$ módulos: de 21×21 (versión 1) a 177×177 (versión 40). A más datos, mayor versión y módulos más pequeños para un mismo tamaño impreso.

Corrección de errores. Los datos se protegen con códigos de Reed–Solomon en cuatro niveles: L, M, Q y H, que permiten recuperar aproximadamente el 7 %, 15 %, 25 % y 30 % de las palabras de código dañadas. Para un respaldo que puede mancharse o rayarse conviene Q o H.

Modos de codificación. El QR elige entre modos numérico, alfanumérico, de bytes y kanji. El modo *alfanumérico* admite 45 caracteres (dígitos, mayúsculas `A`–`Z`, espacio y `$ % * + - . / :`) y codifica cada par de caracteres en 11 bits, es decir 5.5 bits por carácter frente a 8 del modo de bytes.

Capacidad. La versión 40 con nivel L admite como máximo 7089 dígitos, 4296 caracteres alfanuméricos o 2953 bytes. Una parte de Shamir de una clave de 16 o 32 bytes ocupa una fracción pequeña de eso.

Consecuencia de diseño: Base32

Base32 usa sólo `A`–`Z` y `2`–`7`, todos dentro del alfabeto alfanumérico del QR. Si el texto de la parte usa Base32 en mayúsculas y separadores `:`, todo el contenido se codifica en modo alfanumérico: Base32 gasta 8 caracteres por cada 5 bytes, pero a 5.5 bits por carácter eso es 44 bits por cada 40 bits de datos, un sobrecosto de sólo 10 %. Base64, en cambio, usa minúsculas y obliga al modo de bytes: 8 bits por cada 6 bits de datos, un sobrecosto de 33 %.

3 Shamir sobre $\mathbb{F}_{2^8}$: un byte a la vez

3.1 Por qué $\mathbb{F}_{2^8}$

La versión clásica del esquema trabaja en $\mathbb{F}_p$ con p primo mayor que el secreto. Para secretos que son cadenas de bytes de cualquier longitud es más cómodo repartir *cada byte por separado* en el campo de 256 elementos $\mathbb{F}_{2^8}$: las partes miden exactamente lo mismo que el secreto, no hay que elegir primos y la aritmética cabe en tablas pequeñas. Es la elección de SLIP-39, de SSKR y de HashiCorp Vault.

Representamos $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(m(x))$ con el polinomio irreducible de AES, $m(x) = x^8 + x^4 + x^3 + x + 1$ (`0x11B`). Cada byte es un polinomio de grado menor que 8 con coeficientes en $\mathbb{F}_2$:

- la **suma** es el XOR bit a bit, y cada elemento es su propio opuesto ($a + a = 0$, de modo que $-a = a$);
- el **producto** es la multiplicación de polinomios reducida módulo $m(x)$;

- el grupo multiplicativo es cíclico de orden 255 y `0x03` lo genera, así que $a \cdot b = g^{\log a + \log b}$ y $a^{-1} = g^{255 - \log a}$.

3.2 Algoritmos

Algoritmo 1 Aritmética de $\mathbb{F}_{2^8}$ con tablas de exponentes y logaritmos

función *ConstruirTablas*
  $x \leftarrow 1$
  para $i \leftarrow 0$ **a** 254 **hacer**
    $\mathrm{EXP}[i] \leftarrow x$; $\mathrm{EXP}[i + 255] \leftarrow x$; $\mathrm{LOG}[x] \leftarrow i$
    $x \leftarrow x \oplus \mathit{Dup}(x)$ ▷ $x \leftarrow 3 \cdot x$
  fin para
fin función

función $\mathit{Dup}(x)$ ▷ multiplicar por 2 (desplazar y reducir)
  $y \leftarrow x \ll 1$
  si $y \geq 256$ **entonces** $y \leftarrow y \oplus$ `0x11B`
  fin si
  devolver y
fin función

función $\mathit{Mul}(a, b)$
  si $a = 0$ **o** $b = 0$ **entonces**
    devolver 0
  fin si
  devolver $\mathrm{EXP}[\mathrm{LOG}[a] + \mathrm{LOG}[b]]$
fin función

función $\mathit{Inv}(a)$ ▷ requiere $a \neq 0$
  devolver $\mathrm{EXP}[255 - \mathrm{LOG}[a]]$
fin función

Como los coeficientes de Lagrange dependen sólo de las abscisas, se calculan una vez y cada byte del secreto cuesta k multiplicaciones y $k - 1$ XOR.

Algoritmo 2 Reparto de Shamir byte a byte sobre $\mathbb{F}_{2^8}$

Entrada: secreto $S = (s_1, \ldots, s_L)$ de L bytes; umbral k; partes n, con $2 \le k \le n \le 255$
Salida: partes (x, Y_x) con $x = 1, \ldots, n$ y $Y_x \in \mathbb{F}_{2^8}^L$

función *RepartirBytes*(S, k, n)
  para $t \leftarrow 1$ **a** L **hacer** ▷ un polinomio independiente por byte
    $a_0 \leftarrow s_t$
    para $j \leftarrow 1$ **a** $k-1$ **hacer** $a_j \leftarrow$ *ByteAleatorio*
    fin para
    para $x \leftarrow 1$ **a** n **hacer**
      $y \leftarrow 0$
      para $j \leftarrow k-1$ **bajando a** 0 **hacer**
        $y \leftarrow \mathit{Mul}(y, x) \oplus a_j$ ▷ Horner
      fin para
      $Y_x[t] \leftarrow y$
    fin para
  fin para
  devolver $\{(x, Y_x)\}_{x=1}^{n}$
fin función

3.3 Ejemplo a mano: un byte, umbral 2 de 3

Ejemplo: el byte `0x53` con $k = 2$, $n = 3$

Sea $s =$ `0x53` y supongamos que el coeficiente aleatorio es $a_1 =$ `0xCA`, de modo que $f(x) =$ `0x53` $\oplus$ `0xCA` $\cdot\, x$.

Un producto paso a paso. `0xCA` $\cdot$ `0x02`: en binario `0xCA` $=$ 11001010; al desplazar se obtiene 1 10010100, que tiene 9 bits, así que se reduce con `0x11B`$=$ 1 00011011: $110010100 \oplus 100011011 = 010001111 =$ `0x8F`.

Partes.

x	`0xCA` $\cdot\, x$	$f(x) =$ `0x53` $\oplus$ `0xCA` $\cdot\, x$	parte
1	`0xCA`	`0x53` $\oplus$ `0xCA`	`0x99`
2	`0x8F`	`0x53` $\oplus$ `0x8F`	`0xDC`
3	`0x45`	`0x53` $\oplus$ `0x45`	`0x16`

Reconstrucción con las partes 1 y 2. En $\mathbb{F}_{2^8}$ la resta es XOR, así que $\lambda_1 = \dfrac{x_2}{x_2 \oplus x_1} = \dfrac{2}{3}$ y $\lambda_2 = \dfrac{x_1}{x_1 \oplus x_2} = \dfrac{1}{3}$. Con $3^{-1} =$ `0xF6` resulta $\lambda_1 =$ `0xF7` y $\lambda_2 =$ `0xF6`. Entonces

$$s = \texttt{0x99} \cdot \texttt{0xF7} \;\oplus\; \texttt{0xDC} \cdot \texttt{0xF6} = \texttt{0xEE} \oplus \texttt{0xBD} = \texttt{0x53}. \;\checkmark$$

Una sola parte no dice nada: para cualquier byte candidato s' existe un único a_1 que produce esa parte.

3.4 Implementación en Python

Algoritmo 3 Reconstrucción byte a byte por Lagrange en $x = 0$

Entrada: k partes (x_j, Y_j) con x_j distintos y no nulos
Salida: el secreto S

función *ReconstruirBytes*$(\{(x_j, Y_j)\}_{j=1}^{k})$
  para $j \leftarrow 1$ **a** k **hacer** ▷ los λ_j se calculan una sola vez
    $num \leftarrow 1$; $den \leftarrow 1$
    para $m \leftarrow 1$ **a** k, $m \neq j$ **hacer**
      $num \leftarrow Mul(num, x_m)$
      $den \leftarrow Mul(den, x_m \oplus x_j)$ ▷ $x_m - x_j = x_m \oplus x_j$
    fin para
    $d \leftarrow Inv(den)$; $\lambda_j \leftarrow Mul(num, d)$
  fin para
  para $t \leftarrow 1$ **a** L **hacer**
    $s_t \leftarrow \bigoplus_{j=1}^{k} Mul(Y_j[t], \lambda_j)$
  fin para
  devolver $(s_1, \ldots, s_L)$
fin función

Listing 1: `gf256.py`: Shamir byte a byte sobre $\mathbb{F}_{2^8}$

```python
"""Shamir byte a byte sobre GF(2^8) con el polinomio de AES x^8+x^4+x^3+x+1."""
import secrets

EXP, LOG = [0] * 510, [0] * 256
x = 1
for i in range(255):                    # 3 es generador del grupo multiplicativo
    EXP[i] = EXP[i + 255] = x
    LOG[x] = i
    x ^= (x << 1) ^ (0x11B if x & 0x80 else 0)   # x <- 3*x
    x &= 0xFF

def mul(a, b):
    return 0 if a == 0 or b == 0 else EXP[LOG[a] + LOG[b]]

def inv(a):
    if a == 0:
        raise ZeroDivisionError("0 no es invertible")
    return EXP[255 - LOG[a]]

def repartir(secreto: bytes, k: int, n: int):
    """Devuelve n partes (x, bytes); cualesquiera k reconstruyen."""
    if not (2 <= k <= n <= 255):
        raise ValueError("se requiere 2 <= k <= n <= 255")
    partes = {x: bytearray() for x in range(1, n + 1)}
    for b in secreto:                          # un polinomio por byte
        coefs = [b] + [secrets.randbelow(256) for _ in range(k - 1)]
        for x in partes:
            y = 0
            for a in reversed(coefs):          # Horner; la suma en GF(2^8) es XOR
                y = mul(y, x) ^ a
            partes[x].append(y)
```

```
    return [(x, bytes(v)) for x, v in partes.items()]

def reconstruir(partes):
    xs = [x for x, _ in partes]
    if 0 in xs or len(set(xs)) != len(xs):
        raise ValueError("abscisas nulas o repetidas")
    lam = []                                  # Lagrange en 0 (en GF(2^8), -x = x)
    for xj in xs:
        num = den = 1
        for xm in xs:
            if xm != xj:
                num = mul(num, xm)
                den = mul(den, xm ^ xj)
        lam.append(mul(num, inv(den)))
    largo = len(partes[0][1])
    salida = bytearray(largo)
    for (_, y), l in zip(partes, lam):
        for t in range(largo):
            salida[t] ^= mul(y[t], l)
    return bytes(salida)

if __name__ == "__main__":
    assert all(mul(a, inv(a)) == 1 for a in range(1, 256))
    assert mul(0x57, 0x83) == 0xC1            # ejemplo de FIPS-197
    s = b"semilla de 32 bytes para prueba!"
    p = repartir(s, 3, 5)
    assert reconstruir([p[0], p[2], p[4]]) == s
    assert reconstruir(p[1:4]) == s
    print("GF(2^8) OK")
```

La prueba `mul(0x57, 0x83) == 0xC1` es el ejemplo de multiplicación del estándar de AES (FIPS-197 [8]), útil para validar que el campo está bien construido.

4 De la parte al código QR

4.1 Qué debe llevar cada QR

Un QR que sólo contiene los bytes de la parte es frágil en la práctica: al cabo de diez años nadie recuerda cuántas partes se necesitan, a qué secreto pertenece cada una ni con qué programa se crearon. Proponemos un formato textual mínimo, `SSQR1`, con todos sus campos en el alfabeto alfanumérico del QR:

`SSQR1 : ID : k : n : x : DATOS : CRC`

`SSQR1` prefijo y versión del formato;
`ID` identificador *aleatorio* de 32 bits que agrupa las partes de un mismo secreto (no debe derivarse del secreto: un hash del secreto permitiría probar contraseñas por fuerza bruta);
`k, n, x` umbral, total de partes e índice;
`DATOS` los bytes de la parte en Base32 sin relleno;
`CRC` CRC-32 del resto del texto, para detectar errores de lectura o de transcripción.

Dos niveles de integridad

El CRC protege *cada QR* contra errores accidentales, pero no contra un custodio malicioso, que puede alterar su parte y recalcular el CRC. Para eso se añaden 4 bytes de SHA-256 del secreto *dentro* del secreto antes de repartirlo: viajan protegidos por el propio esquema, no revelan nada con menos de k partes y, al reconstruir, una parte alterada hace fallar la comprobación con probabilidad $1 - 2^{-32}$. Es la misma idea que el *digest share* de SLIP-39 y SSKR.

Algoritmo 4 Emisión de las partes como códigos QR

Entrada: secreto S; umbral k; partes n; nivel de corrección $e \in \{\mathrm{L}, \mathrm{M}, \mathrm{Q}, \mathrm{H}\}$
Salida: n imágenes QR

función *EmitirQR*(S, k, n, e)
  $ID \leftarrow$ 4 bytes aleatorios en hexadecimal
  $S' \leftarrow S \,\|\, \mathrm{SHA256}(S)[0..3]$ ▷ suma de verificación interna
  para cada (x, Y_x) **en** *RepartirBytes*(S', k, n) **hacer**
    $c \leftarrow$ "`SSQR1:`ID`:`k`:`n`:`x`:`" $\|$ Base32(Y_x)
    $t \leftarrow c \,\|$ "`:`" $\|$ CRC32(c)
    generar un QR de t con nivel e y guardarlo como imagen
  fin para
fin función

Algoritmo 5 Lectura de los QR y reconstrucción verificada

Entrada: imágenes de t códigos QR
Salida: el secreto S o un error explicado

función *LeerQR*(imágenes)
  para cada imagen **hacer**
    $t \leftarrow$ *DecodificarQR*(imagen)
    separar t en campos; **si** el CRC no coincide **entonces error:** parte dañada
  fin para
  si los ID no coinciden **entonces**
    error: partes de secretos distintos
  fin si
  si hay menos de k partes **entonces**
    error: partes insuficientes
  fin si
  $S' \leftarrow$ *ReconstruirBytes*(partes)
  $S \leftarrow S'[0..L-5]$; $h \leftarrow S'[L-4..L-1]$
  si $\mathrm{SHA256}(S)[0..3] \neq h$ **entonces**
    error: alguna parte es incorrecta
  fin si
  devolver S
fin función

4.2 Implementación

El módulo siguiente usa `segno` [13] para generar los QR (Python puro, sin dependencias) y `zxing-cpp` [14] para leerlos.

Listing 2: `ssqr.py`: formato, generación y lectura de partes en QR

```python
"""Partes de Shamir en códigos QR: formato textual, generación y lectura."""
import base64, hashlib, secrets, zlib
import segno                      # generación de QR (Python puro)
import zxingcpp                   # lectura de QR
from PIL import Image
import gf256

PREFIJO = "SSQR1"

def _b32(b: bytes) -> str:        # Base32 sin relleno: cabe en modo alfanumérico
    return base64.b32encode(b).decode().rstrip("=")

def _de_b32(s: str) -> bytes:
    return base64.b32decode(s + "=" * (-len(s) % 8))

def codificar_parte(ident: str, k: int, n: int, x: int, datos: bytes) -> str:
    cuerpo = f"{PREFIJO}:{ident}:{k}:{n}:{x}:{_b32(datos)}"
    return f"{cuerpo}:{zlib.crc32(cuerpo.encode()):08X}"

def decodificar_parte(texto: str):
    *campos, crc = texto.strip().split(":")
    cuerpo = ":".join(campos)
    if campos[0] != PREFIJO or f"{zlib.crc32(cuerpo.encode()):08X}" != crc:
        raise ValueError("parte dañada o de otro formato")
    _, ident, k, n, x, datos = campos
    return ident, int(k), int(n), int(x), _de_b32(datos)

def repartir_en_qr(secreto: bytes, k: int, n: int, carpeta=".", error="Q"):
    """Agrega una suma de verificación DENTRO del secreto, reparte y genera n PNG."""
    ident = secrets.token_hex(4).upper()        # aleatorio: no depende del secreto
    protegido = secreto + hashlib.sha256(secreto).digest()[:4]
    archivos = []
    for x, y in gf256.repartir(protegido, k, n):
        texto = codificar_parte(ident, k, n, x, y)
        qr = segno.make(texto, error=error, boost_error=False)
        nombre = f"{carpeta}/parte_{ident}_{x}de{n}.png"
        qr.save(nombre, scale=8, border=4)
        archivos.append(nombre)
    return ident, archivos

def leer_qr(archivo: str) -> str:
    res = zxingcpp.read_barcodes(Image.open(archivo))
    if not res:
        raise ValueError(f"no se encontró un QR en {archivo}")
    return res[0].text

def reconstruir_de_qr(archivos):
    partes = [decodificar_parte(leer_qr(a)) for a in archivos]
```

```
    idents = {p[0] for p in partes}
    k = partes[0][1]
    if len(idents) != 1:
        raise ValueError("las partes pertenecen a secretos distintos")
    if len(partes) < k:
        raise ValueError(f"se necesitan {k} partes y hay {len(partes)}")
    protegido = gf256.reconstruir([(p[3], p[4]) for p in partes])
    secreto, chk = protegido[:-4], protegido[-4:]
    if hashlib.sha256(secreto).digest()[:4] != chk:
        raise ValueError("la reconstrucción no verifica: alguna parte es incorrecta")
    return secreto
```

Listing 3: `demo.py`: reparto 3 de 5 en imágenes y reconstrucción

```
import os, glob
from ssqr import repartir_en_qr, reconstruir_de_qr

os.makedirs("partes", exist_ok=True)
secreto = "clave maestra: Tlaxcala-2026!".encode()
ident, archivos = repartir_en_qr(secreto, k=3, n=5, carpeta="partes")
print("Identificador:", ident)
for a in archivos:
    print(" ", a)

# Tres custodios cualesquiera escanean sus códigos
recuperado = reconstruir_de_qr([archivos[0], archivos[2], archivos[4]])
print("Secreto:", recuperado.decode())

# Dos partes no bastan
try:
    reconstruir_de_qr(archivos[:2])
except ValueError as e:
    print("Con dos partes:", e)
```

Una ejecución produce, por ejemplo, el texto siguiente para la parte 1:

```
SSQR1:84FF785B:3:5:1:DH7GFT4KMTUELYPDUW7IMUQDOHPXLLSVWDP6I3QGNHXU5K7GRUI4Q:388DFCF0
```

4.3 Ejemplo ilustrativo

La Figura 1 muestra las cinco partes de un reparto $(3, 5)$ del secreto `clave maestra: Tlaxcala-2026!` con nivel de corrección H, tal como las generó el programa. Cada una es un QR versión 6 (41×41 módulos). Las cinco lucen igual de "aleatorias" y ninguna, ni dos de ellas juntas, revelan algo del secreto.

Tolerancia al daño. En la Figura 2 la parte 3 tiene una mancha que cubre una región central importante del símbolo. Gracias al nivel H, `zxing-cpp` la lee sin problema y el secreto se reconstruye con las partes 1, 3 (manchada) y 5. Las pruebas también confirmaron los dos controles de integridad: una parte alterada por un "custodio" con el CRC recalculado pasa la lectura del QR, pero la reconstrucción la rechaza al fallar la suma de verificación interna.



Parte 1 Parte 2 Parte 3 Parte 4 Parte 5

Figura 1: Las cinco partes de un reparto $(3, 5)$, nivel de corrección H. Cualesquiera tres reconstruyen el secreto.

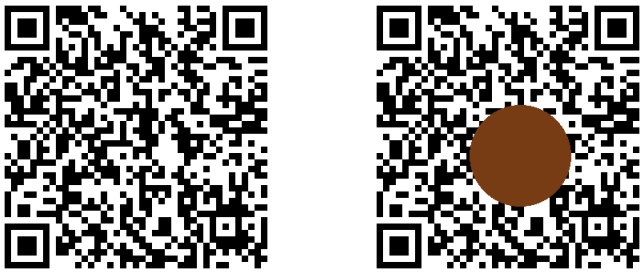


Figura 2: La parte 3 intacta y con una mancha. Con nivel H sigue siendo legible.

4.4 ¿Qué tan grande sale el QR?

El Cuadro 1 da la versión de QR necesaria para una parte en formato `SSQR1` según el tamaño del secreto y el nivel de corrección, calculada con `segno`. Los secretos típicos (claves AES de 16 bytes, semillas de 32 bytes) caben en versiones 3 a 7, fáciles de imprimir y de leer incluso con cámaras modestas.

Tabla 1: Versión del QR para una parte `SSQR1` (modo alfanumérico).

Secreto	Caracteres	L	M	Q	H
16 bytes	62	3	4	4	5
32 bytes	88	4	4	6	7
64 bytes	139	5	6	8	9
128 bytes	242	8	9	11	13
256 bytes	446	11	13	16	18
512 bytes	856	17	19	23	26

5 Secretos largos: el modo híbrido

Para documentos de cientos de bytes (instrucciones de acceso, una clave privada RSA en formato PEM, un archivo de configuración), repartirlo todo produce un QR grande por custodio. La solución es la misma que en el documento anterior sobre el esquema de Shamir: cifrar el documento con AES-GCM bajo una clave aleatoria de 16 bytes, repartir sólo la clave y poner el criptograma en un QR "de bóveda" que puede copiarse sin riesgo y entregarse a todos o guardarse aparte.

Listing 4: `hibrido.py`: documento cifrado en un QR público y partes pequeñas

```python
"""Secretos largos: el documento cifrado va en un QR público; los QR de
los custodios sólo llevan partes de la clave AES."""
import segno, secrets
from Crypto.Cipher import AES
from ssqr import codificar_parte, decodificar_parte, _b32, _de_b32
import gf256

def proteger(documento: bytes, k: int, n: int):
    clave = secrets.token_bytes(16)
    c = AES.new(clave, AES.MODE_GCM)
    ct, tag = c.encrypt_and_digest(documento)
    boveda = "SSQRV:" + _b32(c.nonce + tag + ct)   # QR público con el cifrado
    ident = secrets.token_hex(4).upper()
    partes = [codificar_parte(ident, k, n, x, y)
              for x, y in gf256.repartir(clave, k, n)]
    return boveda, partes

def recuperar(boveda: str, textos):
    datos = _de_b32(boveda.split(":", 1)[1])
    nonce, tag, ct = datos[:16], datos[16:32], datos[32:]
    partes = [decodificar_parte(t) for t in textos]
    clave = gf256.reconstruir([(p[3], p[4]) for p in partes])
    # GCM verifica la integridad: una parte alterada produce ValueError
    return AES.new(clave, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)

if __name__ == "__main__":
    doc = ("Instrucciones de acceso a la bóveda: " + "x" * 400).encode()
    boveda, partes = proteger(doc, 2, 3)
    print("QR bóveda: versión", segno.make(boveda, error="m").version,
          "| QR de parte: versión", segno.make(partes[0], error="m").version)
    assert recuperar(boveda, [partes[0], partes[2]]) == doc
    print("híbrido OK")
```

Con un documento de unos 440 bytes, el QR de bóveda queda en versión 18 y cada QR de parte en versión 3. El modo GCM aporta además la verificación de integridad: si alguna parte presentada es incorrecta, el descifrado falla en lugar de entregar basura. Este es exactamente el diseño de la herramienta Banana Split y de los "sobres" de SSKR (Sección 7).

6 Cuando el QR es el secreto: criptografía visual

6.1 La idea de Naor y Shamir

En 1994 Moni Naor y Adi Shamir propusieron la *criptografía visual* [2]: repartir una imagen en blanco y negro en transparencias de ruido, de modo que al superponerlas físicamente el ojo vea la imagen, sin cálculo alguno. Un QR es una imagen binaria ideal para esto: el secreto es el QR mismo (por ejemplo, una URL con un código de acceso) y nadie puede leerlo con menos sombras de las necesarias.

6.2 Esquema (n, n) por XOR

La versión digital más simple trabaja módulo a módulo sobre la matriz binaria M del QR. Se eligen $n-1$ matrices aleatorias uniformes $S_1, \ldots, S_{n-1}$ y se define $S_n = M \oplus S_1 \oplus \cdots \oplus S_{n-1}$. Cada sombra, y cualquier conjunto de $n-1$ de ellas, es ruido uniforme e independiente de M (es el cifrado de Vernam en dos dimensiones); el XOR de las n sombras devuelve M exactamente.

Algoritmo 6 Criptografía visual (n, n) por XOR sobre la matriz de un QR

Entrada: matriz binaria M de $h \times w$ módulos; número de sombras $n \geq 2$
Salida: sombras $S_1, \ldots, S_n$

función *RepartirXOR*(M, n)
    $S_n \leftarrow M$
    para $i \leftarrow 1$ **a** $n-1$ **hacer**
        $S_i \leftarrow$ matriz $h \times w$ de bits aleatorios uniformes
        $S_n \leftarrow S_n \oplus S_i$
    fin para
    devolver $(S_1, \ldots, S_n)$
fin función

función *CombinarXOR*$(S_1, \ldots, S_n)$
    devolver $S_1 \oplus \cdots \oplus S_n$
fin función



QR secreto | Sombra 1 | Sombra 2 | Sombra 3 | $S_1 \oplus S_2 \oplus S_3$

Figura 3: Esquema $(3, 3)$ por XOR. Las sombras son ruido; su XOR reproduce el QR módulo a módulo y `zxing-cpp` lo lee. Con dos sombras el resultado sigue siendo ruido.

El XOR no es una operación "física": superponer transparencias equivale a un OR (tinta sobre tinta), no a un XOR. Por eso este esquema se combina en una computadora o en un teléfono, no a simple vista.

6.3 Esquema $(2, 2)$ de Naor y Shamir: superposición física

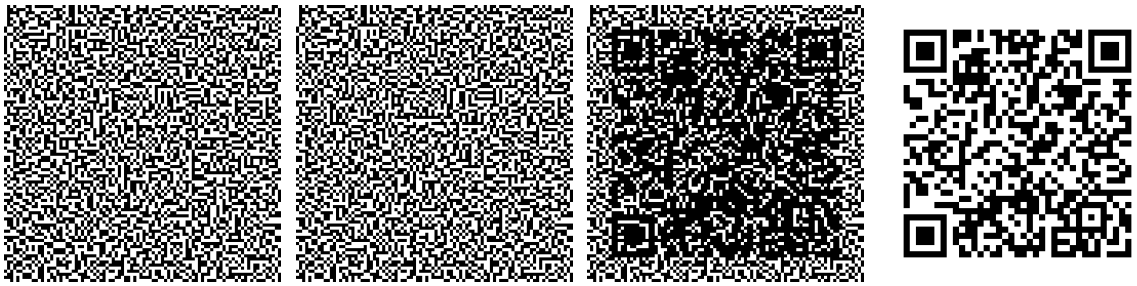
Para que la superposición física funcione, cada módulo se expande en un bloque de 2×2 subpíxeles. Se elige al azar uno de los seis patrones con dos subpíxeles negros. Si el módulo es blanco, ambas transparencias reciben el *mismo* patrón; si es negro, reciben patrones *complementarios*. Al superponerlas, un módulo negro queda totalmente negro (4 de 4 subpíxeles) y uno blanco queda a medio gris (2 de 4). Cada transparencia, sola, es una colección uniforme de bloques con dos subpíxeles negros: no contiene información sobre M.

Algoritmo 7 Criptografía visual $(2, 2)$ de Naor y Shamir

Entrada: matriz binaria M de $h \times w$; $\mathcal{P}$ = los seis patrones 2×2 con dos negros
Salida: transparencias A, B de $2h \times 2w$ subpíxeles

función *RepartirNS*(M)
  para cada módulo (i, j) **hacer**
    $P \leftarrow$ elemento aleatorio uniforme de $\mathcal{P}$
    A[bloque (i, j)] $\leftarrow P$
    si $M[i, j] = 0$ **entonces** B[bloque (i, j)] $\leftarrow P$ ▷ blanco: mismo patrón
    si no B[bloque (i, j)] $\leftarrow \overline{P}$ ▷ negro: complemento
    fin si
  fin para
  devolver (A, B)
fin función

función *Recuperar*$(T = A \vee B)$ ▷ para un lector automático
  devolver $M'[i, j] = 1$ si el promedio del bloque (i, j) de T es mayor que $3/4$
fin función



Transparencia A | Transparencia B | $A \vee B$ (superpuestas) | Tras promediar 2×2

Figura 4: Esquema $(2, 2)$ de Naor y Shamir aplicado a un QR. La superposición muestra el QR con contraste reducido; tras promediar bloques y umbralizar se recupera exactamente.

Lo que observamos en las pruebas

El ojo humano percibe claramente el QR en la superposición (Figura 4), pero `zxing-cpp` *no* decodificó la imagen superpuesta cruda, ni siquiera tras desenfocarla: el contraste de 50 % entre gris y negro y la textura de los subpíxeles confunden su binarizador. Tras promediar cada bloque 2×2 y umbralizar en $3/4$ (función `recuperar_modulos`), la lectura es inmediata. Con transparencias reales, por tanto, conviene fotografiar la superposición y aplicar ese paso, o usar las variantes con reconstrucción perfecta de módulos mencionadas abajo.

Listing 5: `visual_qr.py`: esquemas XOR y de Naor–Shamir sobre la matriz de un QR

```python
"""Criptografía visual aplicada a la matriz de módulos de un código QR."""
import secrets
import numpy as np
import segno

def matriz_qr(texto, error="H", borde=4):
    """Matriz binaria (1 = módulo negro) con zona de silencio incluida."""
```

```
    m = np.array(segno.make(texto, error=error).matrix, dtype=np.uint8)
    return np.pad(m, borde)

# --- (a) Esquema (n,n) por XOR: perfecto y sin expansión de píxeles
def repartir_xor(M, n):
    sombras = [np.frombuffer(secrets.token_bytes(M.size), np.uint8)
               .reshape(M.shape) & 1 for _ in range(n - 1)]
    ultima = M.copy()
    for S in sombras:
        ultima ^= S
    return sombras + [ultima]

def combinar_xor(sombras):
    R = np.zeros_like(sombras[0])
    for S in sombras:
        R ^= S
    return R

# --- (b) Esquema (2,2) de Naor-Shamir: superposición física (OR)
PATRONES = [np.array(p, np.uint8).reshape(2, 2) for p in
            ([1,1,0,0], [0,0,1,1], [1,0,1,0], [0,1,0,1], [1,0,0,1], [0,1,1,0])]

def repartir_naor_shamir(M):
    h, w = M.shape
    A = np.zeros((2*h, 2*w), np.uint8)
    B = np.zeros_like(A)
    for i in range(h):
        for j in range(w):
            p = PATRONES[secrets.randbelow(6)]
            A[2*i:2*i+2, 2*j:2*j+2] = p
            # módulo blanco: mismo patrón; módulo negro: patrón complementario
            B[2*i:2*i+2, 2*j:2*j+2] = p if M[i, j] == 0 else 1 - p
    return A, B

def superponer(A, B):
    return A | B                        # tinta sobre tinta: OR lógico

def recuperar_modulos(S):
    """Promedia bloques 2x2: negro -> 4/4, blanco -> 2/4; umbral 3/4."""
    h, w = S.shape
    prom = S.reshape(h//2, 2, w//2, 2).mean(axis=(1, 3))
    return (prom > 0.75).astype(np.uint8)
```

Listing 6: `demo_visual.py`: comprobación con el lector `zxing-cpp`

```
import numpy as np, zxingcpp
from PIL import Image
from visual_qr import (matriz_qr, repartir_xor, combinar_xor,
                       repartir_naor_shamir, superponer, recuperar_modulos)

def a_imagen(M, escala=8):
    return Image.fromarray(((1 - M) * 255).astype(np.uint8)).resize(
        (M.shape[1]*escala, M.shape[0]*escala), Image.NEAREST)

```

```python
def leer(M, escala=8):
    r = zxingcpp.read_barcodes(a_imagen(M, escala))
    return r[0].text if r else None

M = matriz_qr("https://math.com.mx/boveda?clave=7F3A-91C2")
print("Original:", leer(M))

S = repartir_xor(M, 3)                       # tres sombras, se requieren las tres
print("Una sombra sola:", leer(S[0]))
print("Dos sombras:", leer(combinar_xor(S[:2])))
print("Las tres (XOR):", leer(combinar_xor(S)))

A, B = repartir_naor_shamir(M)
T = superponer(A, B)
print("Transparencia A sola:", leer(A, 4))
print("Superposición cruda:", leer(T, 4))
print("Superposición + promedio 2x2:", leer(recuperar_modulos(T)))
```

La salida del programa confirma que ninguna sombra sola, ni dos de tres, se lee como QR, y que la combinación completa sí:

```
Original: https://math.com.mx/boveda?clave=7F3A-91C2
Una sombra sola: None
Dos sombras: None
Las tres (XOR): https://math.com.mx/boveda?clave=7F3A-91C2
Transparencia A sola: None
Superposición cruda: None
Superposición + promedio 2x2: https://math.com.mx/boveda?clave=7F3A-91C2
```

6.4 Esquemas (k, n) y QR con portada

Los dos esquemas anteriores exigen *todas* las sombras. Naor y Shamir dieron también construcciones (k, n) generales, a costa de mayor expansión de píxeles y menor contraste. La investigación posterior se ha concentrado en tres mejoras específicas para QR:

- **Reconstrucción perfecta de módulos.** Liu, Yan y Pan [7] proponen un esquema visual a color que opera sobre módulos completos en lugar de píxeles, de modo que la superposición de todas las partes produce un QR que un lector estándar decodifica, con una extensión a umbral general.
- **Partes que son QR legibles.** Chow, Susilo y coautores [4] aprovechan la corrección de errores Reed–Solomon: cada parte es un QR válido construido a partir de un QR de portada, que al escanearse muestra un mensaje inocuo, mientras que el secreto sólo aparece al combinar las partes. La misma idea de esconder información en la redundancia del código sustenta los "QR encubiertos" de Chow, Susilo y Baek [5], en los que cualquiera lee el contenido público pero sólo un usuario autorizado recupera el secreto.
- **Distribución por canales públicos.** En la propuesta de Chow *et al.* [6], la parte de cada participante se cifra con su clave pública y se oculta en un QR de apariencia ordinaria, de manera que los QR pueden circular por canales públicos y se necesita un número umbral de participantes para recuperar el secreto.

Visual no es lo mismo que práctico

La criptografía visual es elegante y excelente para docencia, pero para custodiar claves reales el enfoque A (Shamir con el QR como contenedor) es preferible: las partes son pequeñas, toleran daño, admiten cualquier umbral (k, n) sin pérdida de contraste y su seguridad está bien entendida. Las construcciones de QR con portada tienen menos escrutinio público y no conviene usarlas sin una revisión cuidadosa.

7 Librerías y herramientas

7.1 Librerías de Python

Todas las del Cuadro 2 se instalaron y se usaron en los programas de este documento.

Tabla 2: Librerías de Python usadas en este documento.

Paquete (`pip`)	**Versión**	**Función**
`segno`	1.6.6	Genera QR en Python puro, sin dependencias; permite fijar el nivel de corrección, la versión y el modo, y expone la matriz de módulos.
`qrcode`	8.2	Generador alternativo, muy difundido; usa Pillow para producir imágenes.
`zxing-cpp`	3.1.1	Enlace a la biblioteca de lectura ZXing en C++. Lee QR desde imágenes de Pillow o arreglos de NumPy; fue el lector de todas las pruebas.
`opencv-python-headless`	5.0	Incluye `cv2.QRCodeDetector`, otra opción para leer QR, útil si ya se trabaja con video o cámara.
`pycryptodome`	4.1.0	AES-GCM para el modo híbrido; su módulo `Crypto.Protocol.SecretSharing` reparte claves de 16 bytes.
`shamir-mnemonic`	0.3.0	SLIP-39 de Trezor: partes como frases de palabras, que también pueden imprimirse en QR.

Para la parte de Shamir pueden usarse también las librerías revisadas en el documento anterior (`shamirs`, `sslib`); aquí se prefirió una implementación propia sobre $\mathbb{F}_{2^8}$ (`gf256.py`) porque es corta, legible y produce partes del mismo tamaño que el secreto.

7.2 Herramientas existentes

Banana Split.
Originada en Parity Technologies, es una página HTML autocontenida que se abre sin conexión. Cifra el secreto con una frase de contraseña generada automáticamente, lo reparte con Shamir y lo imprime como n códigos QR; la frase se escribe a mano en cada hoja. Para recuperar, se escanea una mayoría de los QR con la cámara y se introduce la frase [11].

SSKR y Gordian SeedTool (Blockchain Commons).
SSKR (*Sharded Secret Key Reconstruction*) reparte un secreto con Shamir sobre $\mathbb{F}_{2^8}$, en uno o dos niveles de grupos con umbrales independientes. La forma recomendada actualmente son los "sobres" SSKR: se cifra el secreto con una clave simétrica nueva y sólo se reparte esa clave, y cada destinatario recibe una copia del sobre cifrado más su parte. Hay implementaciones en C, Rust, Swift, TypeScript y Java, y la aplicación Gordian SeedTool para iOS [10].

ShardQuorum.
Aplicación de Android sin conexión que implementa SSKR sobre $\mathbb{F}_{2^8}$ con un diseño híbrido (reparte una clave aleatoria de 256 bits y guarda el secreto cifrado), verifica la reconstrucción con un *digest share* HMAC-SHA256 y codifica las partes en formato UR (`ur:sskr/`) para QR, con CRC-32 contra errores de transcripción. Incluye una página HTML de recuperación para no depender de la aplicación [12].

8 Aplicaciones

8.1 Aplicaciones en uso

- **Respaldo de semillas de criptomonedas.** Es el uso dominante: Banana Split, SSKR y SLIP-39 producen partes que se imprimen, se laminan o se graban, y se reparten entre familiares o lugares distintos.
- **Respaldo de contraseñas maestras.** El gestor de contraseñas es un punto único de falla: si su titular muere o queda incapacitado, la familia pierde el acceso. ShardQuorum se diseñó para este caso.
- **Herencia digital.** El titular reparte con umbral, por ejemplo, 2 de 3 entre su cónyuge, un notario y un hermano: ninguno puede actuar solo en vida del titular, pero dos de ellos pueden recuperar el acceso cuando sea necesario. Los niveles de grupos de SSKR y SLIP-39 permiten reglas más finas.

8.2 Aplicaciones posibles

Las siguientes se desprenden directamente de las técnicas de este documento; algunas ya se practican con otros soportes.

- **Acceso de emergencia (*break-glass*) en una empresa o una universidad.** La contraseña del administrador raíz o la clave de un servidor crítico se reparte $(3, 5)$ en QR guardados en sobres sellados en cajas fuertes de distintos responsables. En una emergencia, tres de ellos escanean sus QR en un equipo aislado. Combinado con el modo híbrido, el QR de bóveda puede contener el procedimiento completo de recuperación.
- **Respaldo del segundo factor (TOTP).** El QR que muestran los servicios al activar la autenticación de dos pasos contiene una URI `otpauth://` con la semilla. Repartir esa URI en partes QR permite recuperar el segundo factor si se pierde el teléfono, sin que una sola copia impresa sea suficiente para un atacante.
- **Claves privadas de clave pública.** La clave de firma de una autoridad certificadora interna o de firma de código cabe en partes QR pequeñas con el modo híbrido. En una ceremonia de llaves, los custodios presentan sus QR ante testigos.
- **Quórum para abrir información sensible.** Los resultados de un concurso, un examen o una votación de comité se publican cifrados en un QR de bóveda; la clave se reparte entre los miembros del jurado y sólo un quórum puede abrirlos en el momento acordado.
- **Docencia.** El esquema visual $(2, 2)$ impreso en acetatos es una demostración memorable de seguridad perfecta: los alumnos ven que cada transparencia es ruido y que el QR aparece al superponerlas.

9 Recomendaciones de seguridad

El papel y la cámara introducen riesgos que no aparecen en el análisis matemático:

1. **El equipo que reparte ve el secreto.** Shamir protege las partes en reposo, no el secreto mientras se reparte o se reconstruye. Usar un equipo sin conexión y, de ser posible, un sistema arrancado desde un medio de sólo lectura.

2. **Nunca usar generadores o lectores de QR en línea.** Las páginas y aplicaciones que generan o leen QR pueden registrar el contenido. Los programas de este documento trabajan completamente en local.

3. **Cuidar el camino de la imagen.** Las impresoras de red y las fotocopiadoras pueden almacenar trabajos; los teléfonos respaldan fotos automáticamente en la nube. Imprimir en una impresora local y no fotografiar las partes con un teléfono sincronizado.

4. **No poner dos partes en el mismo lugar.** Lo que protege el umbral es la separación física o personal de las partes.

5. **Los metadatos son visibles.** El formato `SSQR1` revela k, n, el índice y la longitud del secreto. Si la longitud es sensible, rellenar el secreto a un tamaño fijo antes de repartir.

6. **Verificar integridad.** Usar la suma de verificación interna o el modo híbrido con AES-GCM: una parte dañada o alterada debe producir un error, no un secreto equivocado.

7. **Elegir nivel Q o H y probar la recuperación.** Antes de distribuir las partes, reconstruir el secreto a partir de ellas una vez. Repetir la prueba periódicamente y reimprimir las partes deterioradas.

8. **Documentar el procedimiento.** Incluir con cada parte, en texto claro, qué es, cuántas partes se necesitan y dónde encontrar el programa o la especificación para recuperar. Es lo que hacen las herramientas de la Sección 7.

Resumen

- Custodia real de claves: Shamir sobre $\mathbb{F}_{2^8}$ con el QR como contenedor, nivel Q o H, suma de verificación interna.
- Secretos largos: modo híbrido (AES-GCM + reparto de la clave).
- Herramientas listas: Banana Split, SSKR (Gordian SeedTool), ShardQuorum.
- Demostraciones y docencia: criptografía visual XOR o de Naor y Shamir.

Referencias

[1] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] M. Naor y A. Shamir, “Visual cryptography,” en *Advances in Cryptology – EUROCRYPT ’94*, LNCS 950, pp. 1–12, Springer, 1995.

[3] ISO/IEC 18004, *Information technology – Automatic identification and data capture techniques – QR Code bar code symbology specification.*

[4] Y.-W. Chow, W. Susilo, G. Yang, J. G. Phillips, I. Pranata y A. M. Barmawi, "Exploiting the error correction mechanism in QR codes for secret sharing," en *Information Security and Privacy (ACISP 2016)*, LNCS 9722, pp. 409–425, Springer, 2016.

[5] Y.-W. Chow, W. Susilo y J. Baek, "Covert QR codes: How to hide in the crowd," en *Information Security Practice and Experience (ISPEC 2017)*, LNCS 10701, pp. 678–693, Springer, 2017.

[6] Y.-W. Chow, W. Susilo, J. Tonien, E. Vlahu-Gjorgievska y G. Yang, "Cooperative secret sharing using QR codes and symmetric keys," *Symmetry*, vol. 10, no. 4, art. 95, 2018.

[7] T. Liu, B. Yan y J.-S. Pan, "Color visual secret sharing for QR code with perfect module reconstruction," *Applied Sciences*, vol. 9, no. 21, art. 4670, 2019.

[8] NIST, *Advanced Encryption Standard (AES)*, FIPS 197, 2001 (actualizado en 2023).

[9] SatoshiLabs, "SLIP-0039: Shamir's Secret-Sharing for Mnemonic Codes." https://github.com/satoshilabs/slips/blob/master/slip-0039.md

[10] Blockchain Commons, "SSKR: Sharded Secret Key Reconstruction." https://developer.blockchaincommons.com/sskr/

[11] Banana Split: respaldos en papel con el esquema de Shamir y códigos QR. https://github.com/kalbroni7/banana_split (bifurcación de `paritytech/banana_split`).

[12] ShardQuorum: reparto de secretos sin conexión para Android. https://github.com/lcdcode/shard-quorum

[13] `segno`: generador de códigos QR en Python. https://pypi.org/project/segno/

[14] `zxing-cpp`: biblioteca de lectura de códigos de barras. https://github.com/zxing-cpp/zxing-cpp